Applied AI

Fine-tuning

Learning Objectives

By the end of the lesson today, you will be able to:

- Explain why transfer learning is usually preferred over training from scratch.
- Build a baseline-to-fine-tune workflow for text classification.
- Run a reproducible Hugging Face `Trainer` workflow.
- Compare baseline and fine-tuned models with metrics and error evidence.

Recap

Over the last few week we have focused on the transformer architecture and existing models from the Hugging Face Hub.

- We compared encoder, decoder, and encoder-decoder families.
- We deconstructed `pipeline()` into tokenizer, model, and decode steps.
- We practiced safe loading habits and explicit model revisions for reproducibility.

This week the flow is:

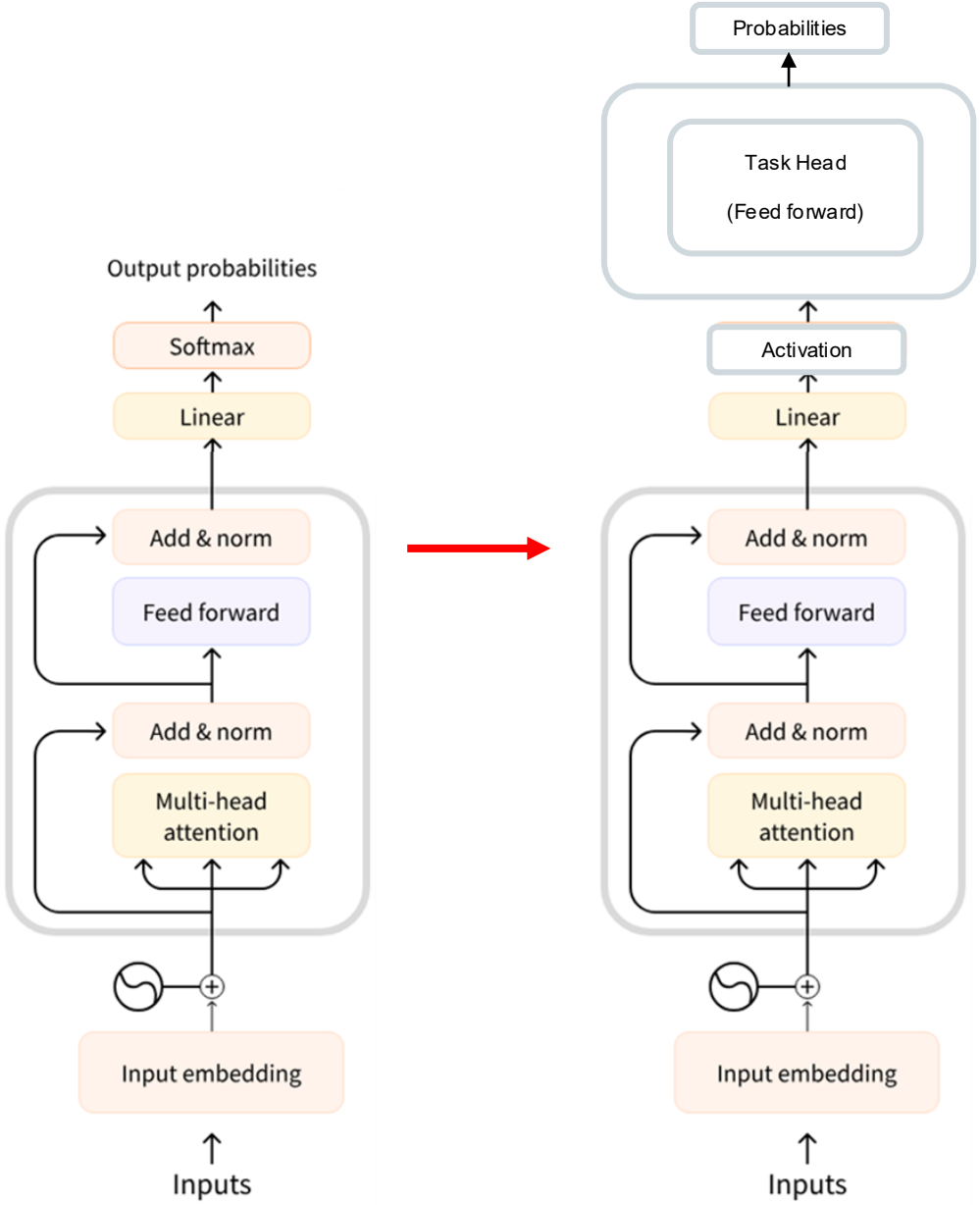
```
architecture intuition -> adaptation workflow ->
evidence -> decision
```

Fine-Tuning

A model can look impressive in a demo and still fail on your real task.

This then prompts us to adapt the model in a process which is called fine-tuning.

Fine-tuning only successful if model has improved at the task.



Fine-Tuning

When fine-tuning we start with a pretrained model.

- Pretraining gives broad language knowledge.
- Fine-tuning aligns that knowledge to your labels and examples.
- The goal is not “train a giant model.”
- The goal is “adapt one responsibly and measure whether it helped.”

Transfer Learning vs Training From Scratch

Transfer learning is usually faster.

It needs less labelled data.

It gives a much more realistic workflow for applied NLP.

Strategy	What you learn	Cost profile
Train from scratch	task + language structure	very high
Fine-tune pretrained model	mainly task boundary	much lower

Generalist to Specialist Adaptation

Think of the pretrained model as a general language encoder.

- Before fine-tuning: broad patterns.
- During fine-tuning: weights shift toward your target labels.
- After fine-tuning: more task-specific behaviour.

Generalist to Specialist Adaptation

Mental model:

Pretraining teaches "language."

Fine-tuning teaches "what matters for this decision."

Lab

To keep the lab feasible and interpretable, we will:

- fine-tune a small encoder model.
- stay on one task and one dataset,
- optimised for `L4` GPU setup

It doesn’t mean bigger models matter, in fact they really do matter!

Baseline-to-Decision Workflow

```
dataset -> baseline -> tokenization -> model
setup -> Trainer run -> evaluation -> error
analysis -> decision
```

- We won’t jump straight to fine-tuning.
- First, we will create a baseline model for comparison
- We will keep the same test split for both models.
- We will end with making a judgement on the models and their performance, not just a training run.

Evidence of Fine-Tuning

A fine-tuned result is only persuasive if it includes:

- A baseline on the same test split,
- Multiple relevant performance metric i.e. accuracy, precision, recall…
- Inspection of failure case,
- A runtime or compute note,
- A final decision statement.

Trainer Mental Model

`Dataset -> Tokenizer -> Data Collator -> Model -> Trainer -> Metrics -> Decision`

`Trainer` automates training loops, evaluation, logging, and checkpointing.

- It does not automate data quality.
- It does not choose good metrics for you.
- It does not interpret your results for you.

Safety and Reproducibility Defaults

Non-negotiables for reliable runs:

- set seeds,
- keep split logic explicit,
- keep tokenizer-model pairing explicit,
- pin checkpoint revision where possible,
- record training arguments and checkpoint path,
- keep `trust_remote_code=False` and `use_safetensors=True`.

Reproducibility defaults

```
SEED = 42
random.seed(SEED)
np.random.seed(SEED)
torch.manual_seed(SEED)
torch.cuda.manual_seed_all(SEED)
sns.set_theme(style='whitegrid')
```

Task and Dataset Choice

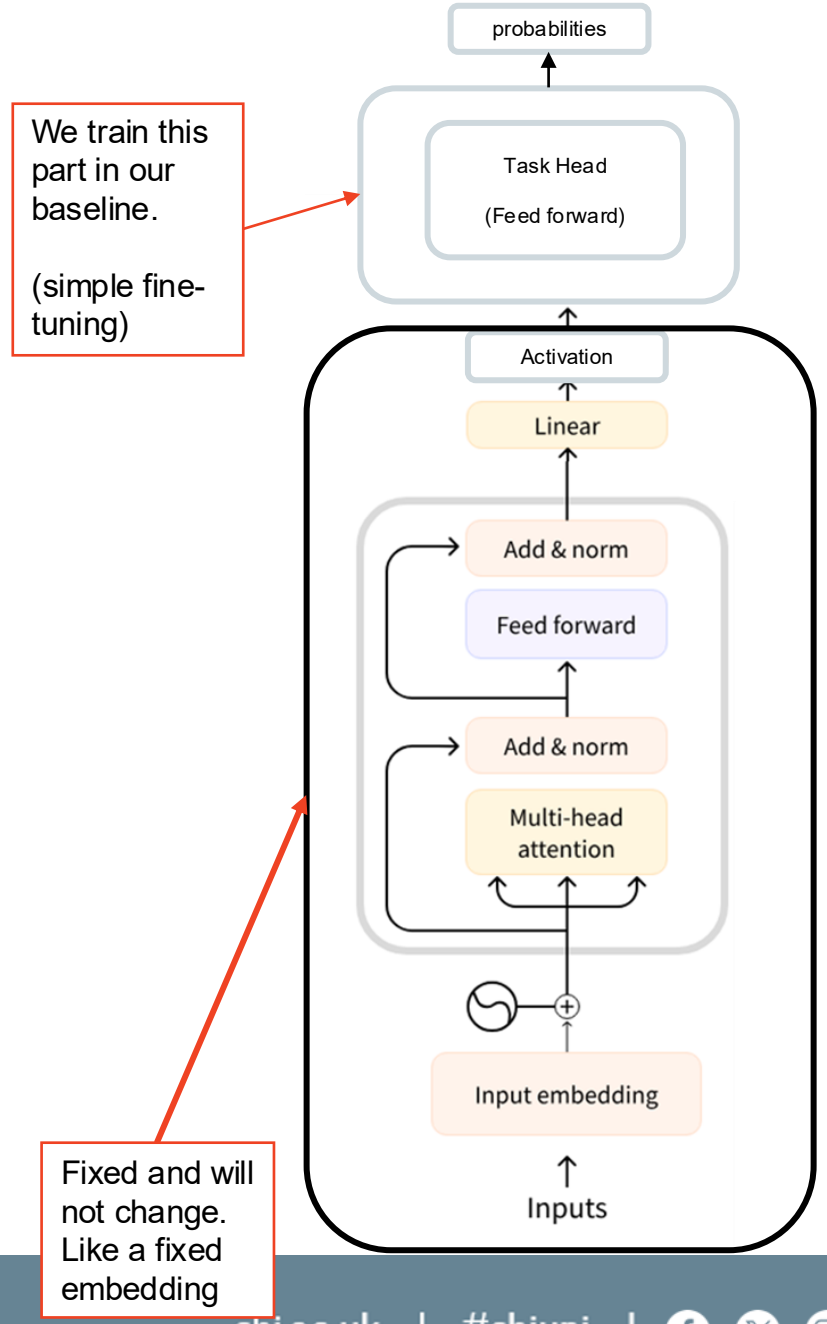
Lab example:

- Dataset: `rotten_tomatoes`
- Baseline: `frozen bert-base-uncased + simple task head`
- Checkpoint: `bert-base-uncased`
- Subsets: `train / val / test`
- Evaluation: same test split for both models

Why Baselines Matter

Fine-tuning should not get a free pass just because it is more modern.

- Baselines are cheaper and easier to debug.
- They tell us whether the task is already fairly easy.
- In our lab we shall create a baseline by fixing the transforms and just training a simple task head.



Reading Baseline Evidence

When you inspect the baseline, do not stop at one metric.

- Check accuracy and metrics
- Look at the confusion matrix.
- Read a few correct predictions.
- Read a few errors before touching the transformer.

Ask:

- What does the baseline already do well?
- Where does it still fail?
- If the baseline is already strong, what extra gain would justify the transformer?

From Raw Text to Transformer Inputs

As we know the transformer cannot train on raw strings directly.

Flow:

```
raw text -> tokenizer -> input_ids +
attention_mask -> collator -> batches -> model
```

- Tokenization converts text into model vocabulary ids.
- Attention masks tell the model which tokens are real and which are padding.
- The collator builds batches with consistent shapes.

Tokenization Micro-Demo

Small example:

- Raw text: `I loved this movie`
- Tokens: `[CLS] i loved this movie [SEP]`
- Attention mask: `1 1 1 1 1 1`

Dynamic Padding

Dynamic padding is a practical efficiency choice.

It matters even for small models.

And becomes even more important as sequence lengths grow.

Padding strategy	What happens	Cost implication
Fixed max length	every batch padded to the same width	more waste
Dynamic padding	batch padded only to local max	less waste

Label Mapping and Model Setup

Keep the label mapping explicit:

- `id2label` maps numeric ids to readable class names,
- `label2id` maps names back to numeric ids,
- the model config should match the dataset schema,
- tokenizer and model must be paired deliberately.

This matters because:

- label mismatch can silently ruin interpretation,
- explicit mapping makes predictions easier to audit.

Performance Metrics: Accuracy

Definition: The proportion of correct predictions (both true positives and true negatives) among the total number of cases examined.

- Formula: Accuracy = (TP + TN) / (TP + TN + FP + FN)

Example: Movie Review Sentiment Analysis

		Predicted Sentiment: Positive	Predicted Sentiment: Negative
Actual Sentiment	Positive	85 (Correctly identified positive reviews)	15 (Negative reviews predicted as positive)
	Negative	10 (Positive reviews predicted as negative)	90 (Correctly identified negative reviews)

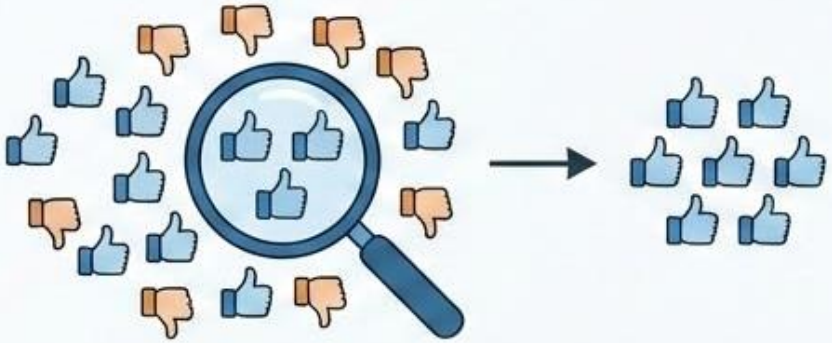
Calculation: Accuracy $= \frac{85 + 90}{85 + 15 + 10 + 90} = \frac{175}{200} = 87.5\%$

Interpretation: The model correctly classified 87.5% of the movie reviews.
Note: Accuracy can be misleading if the dataset is highly imbalanced (e.g., 95% positive reviews).

Performance Metrics: Precision & Recall
(Sentiment Analysis Context)

PRECISION (The "Exactness")

Of all reviews predicted as Positive, how many were actually Positive?



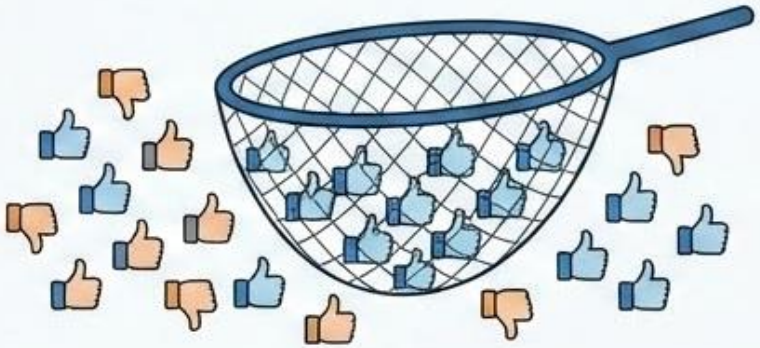
High Precision means few False Positives (junk labeled as good).

Example: The model rarely labels a negative review as positive, but might miss some true positives.

$$\text{Precision} = \frac{\text{True Positives}}{(\text{True Positives} + \text{False Positives})}$$

RECALL (The "Completeness")

Of all actual Positive reviews, how many did the model correctly identify?



High Recall means few False Negatives (good stuff missed).

Example: The model finds almost all positive reviews, even if it incorrectly labels some negative ones as positive.

$$\text{Recall} = \frac{\text{True Positives}}{(\text{True Positives} + \text{False Negatives})}$$

Performance Metrics: F1-Score

What is F1-Score?

- Harmonic mean of Precision and Recall.
- Balances both metrics into a single value.
- Useful when classes are imbalanced.

$$F1 = 2 * \frac{(\text{Precision} * \text{Recall})}{(\text{Precision} + \text{Recall})}$$

Context: Movie Review Sentiment

Scenario: Classifying reviews as Positive or Negative.

Precision (e.g., for 'Negative'): Of all reviews detected as negative, how many were truly negative? (Minimizes False Negatives)

Recall (e.g., for 'Negative'): Of all actual negative reviews, how many were correctly detected? (Minimizes False Positives)

F1-Score gives a balanced view, crucial if one sentiment (e.g., Negative) is rare.

Performance Metrics: Macro F1

For each class we can calculate the Precision, Recall and F1-score for that class.

The macro F1 score is the average of the F1 scores across all classes (unweighted).

Accuracy vs Macro F1

Accuracy is easy to read.

Macro F1 is harder to game when class performance is uneven.

For this lab, both belong in the final comparison.

If they disagree, inspect class-level behaviour before drawing conclusions.

Metric	What it tells us	Risk if used alone
Accuracy	overall fraction correct	can hide weak class performance
Macro F1	balance across classes	less intuitive for beginners

Training Arguments That Matter

```
training_args = TrainingArguments(
    output_dir='assets/full_tune',
    learning_rate=2e-5,
    per_device_train_batch_size=16,
    per_device_eval_batch_size=32,
    num_train_epochs=2,
    weight_decay=0.01,
    eval_strategy='epoch',
    save_strategy='epoch',
    logging_steps=50,
    seed=SEED,
    report_to='none',
    load_best_model_at_end=True,
    metric_for_best_model='macro_f1',
    greater_is_better=True,)
```

This is the folder where Trainer writes outputs such as checkpoints and model predictions.

Training Arguments That Matter

```
training_args = TrainingArguments(
    output_dir='assets/full_tune',
    learning_rate=2e-5,
    per_device_train_batch_size=16,
    per_device_eval_batch_size=32,
    num_train_epochs=2,
    weight_decay=0.01,
    eval_strategy='epoch',
    save_strategy='epoch',
    logging_steps=50,
    seed=SEED,
    report_to='none',
    load_best_model_at_end=True,
    metric_for_best_model='macro_f1',
    greater_is_better=True,)
```

This is the optimizer's initial learning rate. It controls how large each parameter update is during training. In Trainer, this is the starting learning rate

Training Arguments That Matter

```
training_args = TrainingArguments(
    output_dir='assets/full_tune',
    learning_rate=2e-5,
    per_device_train_batch_size=16,
    per_device_eval_batch_size=32,
    num_train_epochs=2,
    weight_decay=0.01,
    eval_strategy='epoch',
    save_strategy='epoch',
    logging_steps=50,
    seed=SEED,
    report_to='none',
    load_best_model_at_end=True,
    metric_for_best_model='macro_f1',
    greater_is_better=True,)
```

This is the training batch size on each device. So if you have one GPU, the batch size is 16. If you had two GPUs, the total batch size would be 32 before any gradient accumulation

Training Arguments That Matter

```
training_args = TrainingArguments(
    output_dir='assets/full_tune',
    learning_rate=2e-5,
    per_device_train_batch_size=16,
    per_device_eval_batch_size=32,
    num_train_epochs=2,
    weight_decay=0.01,
    eval_strategy='epoch',
    save_strategy='epoch',
    logging_steps=50,
    seed=SEED,
    report_to='none',
    load_best_model_at_end=True,
    metric_for_best_model='macro_f1',
    greater_is_better=True,)
```

This is the evaluation batch size on each device. You often make this larger than the training batch size because evaluation does not need backpropagation, so it usually uses less memory.

Training Arguments That Matter

```
training_args = TrainingArguments(
    output_dir='assets/full_tune',
    learning_rate=2e-5,
    per_device_train_batch_size=16,
    per_device_eval_batch_size=32,
    num_train_epochs=2,
    weight_decay=0.01,
    eval_strategy='epoch',
    save_strategy='epoch',
    logging_steps=50,
    seed=SEED,
    report_to='none',
    load_best_model_at_end=True,
    metric_for_best_model='macro_f1',
    greater_is_better=True,)
```

This tells Trainer to make two full passes through the training dataset. One epoch means seeing the whole training set once.

Training Arguments That Matter

```
training_args = TrainingArguments(
    output_dir='assets/full_tune',
    learning_rate=2e-5,
    per_device_train_batch_size=16,
    per_device_eval_batch_size=32,
    num_train_epochs=2,
    weight_decay=0.01,
    eval_strategy='epoch',
    save_strategy='epoch',
    logging_steps=50,
    seed=SEED,
    report_to='none',
    load_best_model_at_end=True,
    metric_for_best_model='macro_f1',
    greater_is_better=True,)
```

This adds L2-style regularization through the optimizer, which helps discourage weights from growing too large and can reduce overfitting. In Trainer this is the optimizer's weight decay value.

Training Arguments That Matter

```
training_args = TrainingArguments(
    output_dir='assets/full_tune',
    learning_rate=2e-5,
    per_device_train_batch_size=16,
    per_device_eval_batch_size=32,
    num_train_epochs=2,
    weight_decay=0.01,
    eval_strategy='epoch',
    save_strategy='epoch',
    logging_steps=50,
    seed=SEED,
    report_to='none',
    load_best_model_at_end=True,
    metric_for_best_model='macro_f1',
    greater_is_better=True,)
```

This means evaluation is run at the end of each epoch, not every fixed number of steps. So after epoch 1 and after epoch 2, Trainer evaluates on eval_dataset

Training Arguments That Matter

```
training_args = TrainingArguments(
    output_dir='assets/full_tune',
    learning_rate=2e-5,
    per_device_train_batch_size=16,
    per_device_eval_batch_size=32,
    num_train_epochs=2,
    weight_decay=0.01,
    eval_strategy='epoch',
    save_strategy='epoch',
    logging_steps=50,
    seed=SEED,
    report_to='none',
    load_best_model_at_end=True,
    metric_for_best_model='macro_f1',
    greater_is_better=True,)
```

This means checkpoints are also saved at the end of each epoch. So your saving schedule matches your evaluation schedule.

That matters because load_best_model_at_end=True requires the save and evaluation strategies to match

Training Arguments That Matter

```
training_args = TrainingArguments(
    output_dir='assets/full_tune',
    learning_rate=2e-5,
    per_device_train_batch_size=16,
    per_device_eval_batch_size=32,
    num_train_epochs=2,
    weight_decay=0.01,
    eval_strategy='epoch',
    save_strategy='epoch',
    logging_steps=50,
    seed=SEED,
    report_to='none',
    load_best_model_at_end=True,
    metric_for_best_model='macro_f1',
    greater_is_better=True,)
```

This controls how often training logs are written when the logging strategy is "steps". The default logging strategy is "steps", so with this setting Trainer logs every 50 update steps. This is mainly for monitoring loss and progress during training; it does not control evaluation or checkpoint saving in your code

Training Arguments That Matter

```
training_args = TrainingArguments(
    output_dir='assets/full_tune',
    learning_rate=2e-5,
    per_device_train_batch_size=16,
    per_device_eval_batch_size=32,
    num_train_epochs=2,
    weight_decay=0.01,
    eval_strategy='epoch',
    save_strategy='epoch',
    logging_steps=50,
    seed=SEED,
    report_to='none',
    load_best_model_at_end=True,
    metric_for_best_model='macro_f1',
    greater_is_better=True,)
```

This sets the random seed at the beginning of training, which helps with reproducibility. It reduces run-to-run randomness, though exact reproducibility can still depend on other details such as random initialization and hardware behaviour.

Training Arguments That Matter

```
training_args = TrainingArguments(
    output_dir='assets/full_tune',
    learning_rate=2e-5,
    per_device_train_batch_size=16,
    per_device_eval_batch_size=32,
    num_train_epochs=2,
    weight_decay=0.01,
    eval_strategy='epoch',
    save_strategy='epoch',
    logging_steps=50,
    seed=SEED,
    report_to='none',
    load_best_model_at_end=True,
    metric_for_best_model='macro_f1',
    greater_is_better=True,)
```

This disables external experiment tracking integrations.

Training Arguments That Matter

```
training_args = TrainingArguments(
    output_dir='assets/full_tune',
    learning_rate=2e-5,
    per_device_train_batch_size=16,
    per_device_eval_batch_size=32,
    num_train_epochs=2,
    weight_decay=0.01,
    eval_strategy='epoch',
    save_strategy='epoch',
    logging_steps=50,
    seed=SEED,
    report_to='none',
    load_best_model_at_end=True,
    metric_for_best_model='macro_f1',
    greater_is_better=True,)
```

At the end of training, Trainer reloads the checkpoint that performed best on your chosen evaluation metric, rather than just keeping the very last checkpoint. This only works when evaluation is enabled, and it requires the save and eval schedules to be aligned.

Training Arguments That Matter

```
training_args = TrainingArguments(
    output_dir='assets/full_tune',
    learning_rate=2e-5,
    per_device_train_batch_size=16,
    per_device_eval_batch_size=32,
    num_train_epochs=2,
    weight_decay=0.01,
    eval_strategy='epoch',
    save_strategy='epoch',
    logging_steps=50,
    seed=SEED,
    report_to='none',
    load_best_model_at_end=True,
    metric_for_best_model='macro_f1',
    greater_is_better=True,)
```

This tells Trainer which evaluation metric to use when deciding what “best model” means. The metric name must match something returned by your evaluation pipeline, typically something produced by compute_metrics.

Training Arguments That Matter

```
training_args = TrainingArguments(
    output_dir='assets/full_tune',
    learning_rate=2e-5,
    per_device_train_batch_size=16,
    per_device_eval_batch_size=32,
    num_train_epochs=2,
    weight_decay=0.01,
    eval_strategy='epoch',
    save_strategy='epoch',
    logging_steps=50,
    seed=SEED,
    report_to='none',
    load_best_model_at_end=True,
    metric_for_best_model='macro_f1',
    greater_is_better=True,)
```

This tells Trainer that larger values of macro_f1 are better. That is the correct choice for F1-based metrics, because higher F1 means better performance.

What Trainer Automates

`Trainer` helps with:

- the training loop,
- evaluation cadence,
- logging,
- checkpoint saving,
- loading the best model at the end.

What Trainer Does Not Automate

You still own:

- label sanity checks,
- metric choice,
- error analysis,
- the final judgement.

`Trainer` reduces boilerplate code needed. It does not replace evaluation discipline.

Running the Fine-Tune

The minimum required when fine-tuning is:

- one clean run through of training with explicit arguments,
- logged validation and test metrics,
- checkpoint path recorded,
- no confusion about what changed from baseline.

Run training

```
train_result = trainer.train()
train_result
```

[1068/1068 01:33, Epoch 2/2]

Epoch	Training Loss	Validation Loss	Accuracy	Macro F1
1	0.328759	0.331809	0.858349	0.858283
2	0.224056	0.402837	0.869606	0.869600

```
Writing model shards:   0%|          | 0/1 [00:00<?, ?it/s]
Writing model shards:   0%|          | 0/1 [00:00<?, ?it/s]
```

Running the Fine-Tune

Checking training:

- During training we are looking for the loss function to go down with the performance metric going up (i.e. macro F1).
- If loss goes down but macro F1 stays flat, something is off.
- Check for label issues, metric mismatch, or a learning rate that is not helping the decision boundary.

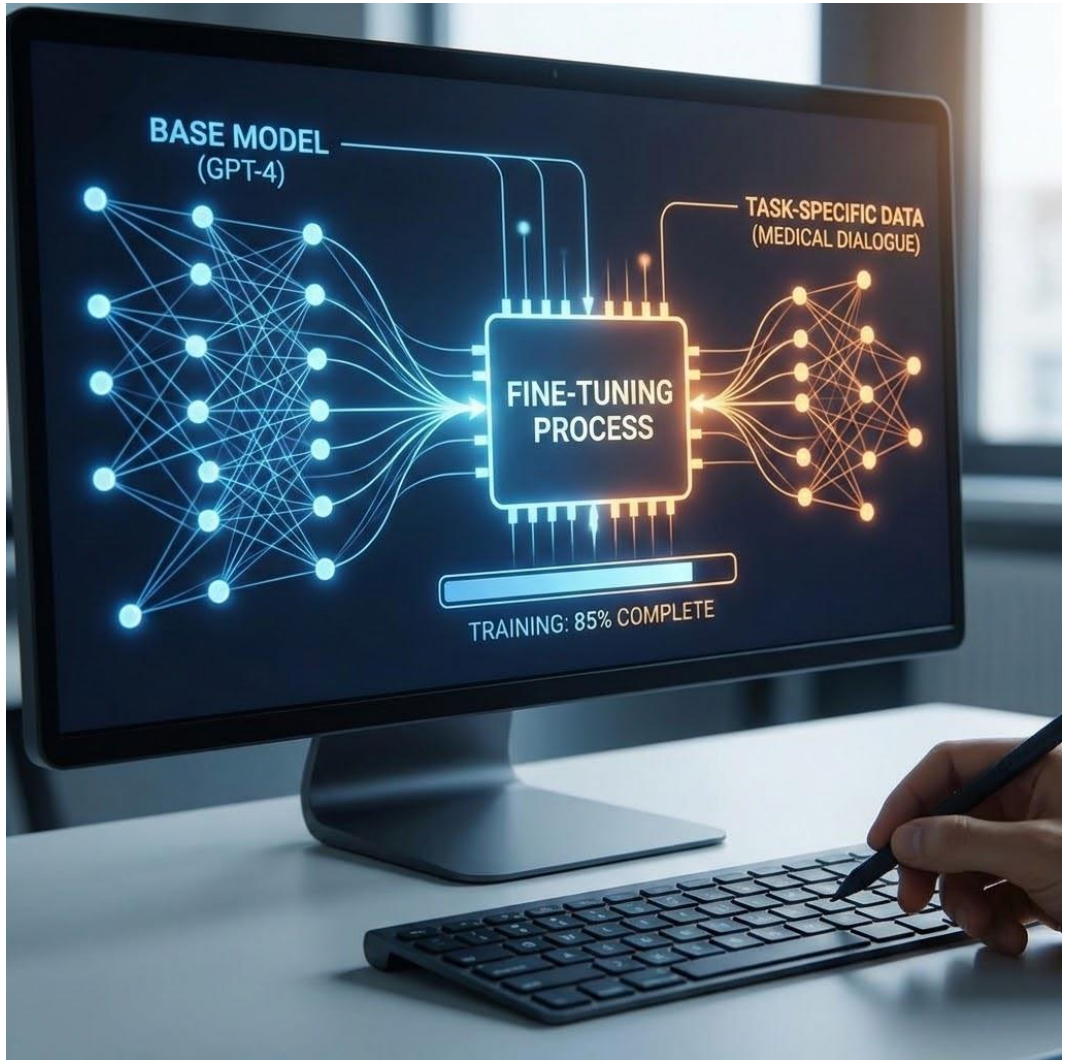
Error Analysis Beyond Metrics

Metrics like accuracy, macro F1 etc are necessary, but not sufficient. We should delve deeper, for instance

- Examine some of the errors
- Look for patterns in the errors, could there be a reason (perhaps linguistic) for the error.
- Can we form a mitigation plan to overcome the errors?

Common Failure Modes

Symptom	Likely cause	First fix
validation metrics flat	learning rate mismatch	sweep around `2e-5`
train improves, test drops	overfitting	stronger regularisation
strange label behaviour	label mapping bug	verify `id2label` / `label2id`
out-of-memory error	batch or sequence too large	reduce batch size or max length

Summary

- fine-tuning is adaptation under constraints, not brute-force retraining,
- baseline + metrics + error analysis belong together,
- `Trainer` saves time, but interpretation is still your job,